

ETC5512: Instruction to Open Data

Lecturer: Kate Saunders

Table of contents

Learning Objectives	1
Before your tutorial	1
Install and load R Packages	2
Exercise 1: Atlas of Living Australia	3
a. Download	3
b. Data quality checks	4
c. Data collection methods	4
Learning to code	4
Practice In Your Own Time	5
Citations	5

Learning Objectives

- Utilise and access open data sources
- Assess the collection methods and the quality of the data
- Write code to conduct quality checks of the data
- Also learn your way around RStudio.

Before your tutorial

! Come prepared!

Ensure you have installed both R and RStudio.

Work through the following [startR modules](#)

- Module 1: Follow these [installation instructions](#)

- Module 2: Learn about [RStudio Basics](#)
- Module 3: Do the module on [R Basics](#)

These should take you ~ 40 minutes.

Install and load R Packages

Run the below code in the R Console to install the packages we need for today's tutorial.

```
install.packages("tidyverse")  
install.packages("galah")
```

Then load the packages using the `library()` function by adding the below code an R Script.

```
library(tidyverse)  
library(galah)
```

💡 Why do we need packages?

Packages in R provide additional functions and tools, for specialised tasks easier - like data visualisation, modeling, or data manipulation.

💡 `install.packages` or `library`: which one do I use?

- Installing packages is like screwing in a lightbulb.
- Loading the package using `library` is like turning the switch on and off.

You only need to screw in the lightbulb once (use `install.packages`), but you any time you want to use the light you need to turn it on (load the package and its functions using `library`).

💡 R Console or R Script - where do I write my code?

An **R Console** is where you run commands interactively and see immediate output — it's useful for quick calculations, testing code, and exploring data.

An **R Script** is a saved file (.R) where you write and store a sequence of commands so they can be edited, reused, shared, and rerun later — making your work reproducible and organised.

Exercise 1: Atlas of Living Australia

The Atlas of Living Australia is a major resource for occurrence data on animals, plants, insects, fish.

a. Download

- i. Point your browser to <https://www.ala.org.au>. Check the terms of use. Does it have a license?
- ii. Using the `galah` library, and the function `occurrences` extract the records for platypus. To download the data from this API **you will need to register with your email** first. Once you've done that change the code below so it has your email.

```
library(galah)

# add your email address below
galah_config(email = "ADD_YOUR_EMAIL_HERE",
              download_reason_id = 10,
              verbose = TRUE)

platypus <- galah_call() |>
  galah_identify("Ornithorhynchus anatinus") |>
  atlas_occurrences()
```

Take a look at the data you downloaded.

```
View(platypus)
```

- iii. Let's do some data wrangling to tidy up our data.

```
platypus <- platypus |>
  rename(Longitude = decimalLongitude,
          Latitude = decimalLatitude) |>
  mutate(eventDate = as.Date(eventDate)) |>
  filter(!is.na(eventDate)) |>
  filter(!is.na(Longitude)) |>
  filter(!is.na(Latitude))
```

- iv. Save our result. To save both your data and the file you are working on in the same place, in the top menu go to Session > Set Working Directory > To Source File Location. (If you are already familiar with R Projects you may like to use that instead.)

```
platypus_file_name = "platypus.csv"
write_csv(platypus, file = platypus_file_name)
```

b. Data quality checks

- i. Plot the locations of sightings. Where in Australia are platypus found?

```
read_csv(platypus_file_name)

ggplot(platypus, aes(x=Longitude, y=Latitude)) +
  geom_point()
```

- ii. What dates of sightings are downloaded?

```
platypus |> select(eventDate) |> summary()
```

c. Data collection methods

How is this data collected? Explain the ways that a platypus sighting would be added to the database. Also think about what might be missing from the data?

Learning to code

Coding tips

When you are getting started you should check your understanding of what each piece of code does. Some tips:

- Use the help menu to look up what functions do and example use e.g. `?occurrences`, `?write_csv`.
- Copy and paste code chunks in Generative AI and ask it to explain what the code is doing. Here are some examples:
 - [Example prompt in ChatGPT asking about occurrences](#)
 - [Example prompt in Claude asking about functions in tidyverse](#)

Then to consolidate your learning, go back and add comments to your code using the `#` to explain what the different lines do.

! Academic Integrity

You must be able to explain any code you submit - it is your work! That doesn't mean you can't use generative AI, but you should not bulk copy code you don't understand. Remember you must also show your AI prompts like above on your assignment.

Practice In Your Own Time

- i. Complete [startR modules 1 - 3](#)
- ii. Complete the Open Data Institute Quizzes from Lectures to consolidate your understanding.
- iii. Review the generative AI prompts above to learn more about coding in R.
- iv. Find examples of the different flavours of open data we discussed in class.
 - Find the license information
 - Look at the meta data.

Citations

💡 Cite packages you use! Here's how ...

```
citation("galah")  
citation("tidyverse")
```

Westgate M, Stevenson M, Kellie D, Newman P (2026). *galah: Biodiversity Data from the GBIF Node Network*. R package version 2.0.2, <https://CRAN.R-project.org/package=galah>

Wickham H, Averick M, Bryan J, Chang W, McGowan LD, François R, Golemund G, Hayes A, Henry L, Hester J, Kuhn M, Pedersen TL, Miller E, Bache SM, Müller K, Ooms J, Robinson D, Seidel DP, Spinu V, Takahashi K, Vaughan D, Wilke C, Woo K, Yutani H (2019). “Welcome to the tidyverse.” *Journal of Open Source Software*, 4(43), 1686. doi:10.21105/joss.01686 <https://doi.org/10.21105/joss.01686>.

© Copyright Monash University